Deep Learning MCQ Cheatsheet for CCEE Exam

Session 17: Introduction to Deep Learning & TensorFlow/Keras

Core Concepts

- **Deep Learning**: Subset of ML using multi-layer neural networks to automatically learn features from data
- **Neural Network Layers**: Input → Hidden layers (multiple) → Output (deeper = more complex patterns)
- **Neuron Computation**: $z = w_1x_1 + w_2x_2 + ... + b$, then $a = \text{activation}(z)$

Key Terms

- **Weights (w)**: Strength of input connections
- **Bias (b)**: Shift term allowing threshold adjustment
- **Activation Function**: Introduces non-linearity for complex pattern learning

Activation Functions

Function	Range	Key Use	Notes
Sigmoid	(0, 1)	Output layers, binary classification	Can suffer from vanishing gradient
Tanh	(-1, 1)	Hidden layers	Zero-centered, preferred over sigmoid
ReLU	[0, ∞)	Hidden layers in modern networks	Fast training, prevents vanishing gradient

TensorFlow & Keras

- **TensorFlow**: Framework with computation graphs and tensors (multidimensional arrays)
- **Keras**: High-level API (part of tf.keras) for easy neural network building
- **Core Keras Workflow**: Load data → Preprocess → Define model → Compile → Fit (train) → Evaluate → Predict
- **Key Keras Components**: Layer (encapsulates weights), Model (container linking layers)

Session 18-19: Neural Networks, Autoencoders & Backpropagation

Single-Layer Neural Network (Perceptron)

- **Structure**: Input layer + Output layer (NO hidden layer)
- **Limitation**: Only solves **linearly separable** problems (cannot solve XOR)
- **Application**: Simple binary classification on separable data

Autoencoders

- **Structure**: Encoder → Bottleneck (latent code) → Decoder
- **Goal**: Reconstruct input from compressed representation
- **Applications**: Dimensionality reduction, denoising, anomaly detection
- **Loss**: Reconstruction error (difference between input and output)

Backpropagation of Errors

- **Definition**: Algorithm to compute gradients of loss w.r.t. all weights using chain rule
- **Process**: Forward pass (compute loss) → Backward pass (propagate error) → Get gradients → Weight update
- **Key Fact**: Backprop computes gradients; gradient descent uses them for updates
- **Requirement**: Works with differentiable activation functions

---

Session 20: Deep Learning Essentials (Regularization)

Early Stopping

- **Mechanism**: Monitor validation loss/accuracy; stop when no improvement for patience epochs
- **Effect**: Prevents overfitting and memorization of training noise
- **Category**: Regularization technique (does not change architecture)
- **Requirement**: Separate validation set

Dropout

- **Mechanism**: Randomly deactivate fraction of neurons during training (e.g., 20-50% rate)
- **Effect**: Prevents co-adaptation, forces robust feature learning
- **Key Point**: Used ONLY during training; all neurons active at inference
- **Category**: Regularization technique

L1 Regularization (Lasso)

- **Penalty**: Sum of absolute values of weights
- **Effect**: Pushes many weights **exactly to zero** → **sparse** models
- **Application**: Feature selection (some features eliminated)

L2 Regularization (Ridge)

- **Penalty**: Sum of squared weights
- **Effect**: Shrinks weights smoothly toward zero (rarely exactly zero)
- **Application**: Distributes weight penalty evenly

L1 vs L2 Summary

Aspect	L1	L2
Penalty	$\Sigma \lvert w \rvert$	Σw^2
Effect	Many weights → 0	Weights shrink smoothly
Use	Feature selection	General complexity control

Stochastic Gradient Descent (SGD) - Single Sample Update

- **Definition**: Update weights after **each individual** training example
- **Formula**: $w_{new} = w_{old} - \eta \cdot (\partial L/\partial w)$ where η = learning rate
- **Memory**: Low (one sample at a time)
- **Updates**: Frequent but noisy
- **With L2**: Extra shrinkage term $-\lambda w$ added
- **With L1**: Term proportional to sign(w) added

Session 21-22: Batch Training & Optimizers

Three Gradient Descent Variants

Variant	Data Per Step	Speed	Stability	Memory	Common
Batch GD	Entire dataset	Slow	Very stable	High	No
SGD	1 sample	Fast	Noisy	Low	Research
Mini-batch GD	Small batch (32-128)	Balanced	Stable	Moderate	**YES**

Classic Backpropagation

- **Role**: Computes gradients via chain rule
- **NOT an optimizer**: Only provides gradient information
- **Paired with**: SGD, momentum, Adam, etc. for actual weight updates

Momentum-Based Backpropagation

- **Mechanism**: Accumulates moving average of past gradients (velocity term)
- **Effect**: Smooths oscillations, speeds up learning along consistent directions
- **Analogy**: Ball rolling downhill keeps momentum, overshoots bumps
- **Advantage**: Faster convergence, reduced zig-zag behavior

ADAM Optimizer (Adaptive Moment Estimation)

- **Mechanism**: Combines:
  - **First moment**: Moving average of past gradients (like momentum)
  - **Second moment**: Moving average of squared gradients (adaptive learning rate)
- **Per-parameter learning**: Each weight gets its own adapted learning rate
- **Advantage**: Works well "out of the box"; less tuning needed
- **Status**: Most widely used default optimizer in deep learning

Optimizer Comparison

Feature	Plain SGD	Momentum	Adam
Uses past gradients	No	Yes	Yes
Adaptive LR	No	No	Yes
Practical use	Limited	Research	**Common**
Tuning	High	Medium	Low

Session 23-24: CNNs, Transfer Learning & Object Detection

PyTorch Basics

- **Core**: Tensors (GPU-ready arrays) + Autograd (automatic differentiation)
- **Model Building**: Extend `torch.nn.Module`, define layers in `__init__`, forward pass in `forward()`
- **Graph Type**: **Dynamic** (define-by-run) - graph built as code executes

PyTorch vs TensorFlow

Aspect	PyTorch	TensorFlow
Graph	Dynamic (define-by-run)	Static (historically)
Style	Pythonic, intuitive	More boilerplate
Debugging	Easier	Harder
Common Use	Research, experimentation	Production, large-scale

Convolutional Neural Networks (CNNs)

- **Core Idea**: Slide small filter/kernel over image to detect local patterns
- **Key Properties**:
  - Local connectivity (each neuron sees small region)
  - Weight sharing (same filter across image)
  - Multiple filters (detect different patterns)
- **Output**: Feature maps showing where patterns appear
- **Best For**: Images, vision tasks (classification, detection, segmentation)

CNN Key Terms

- **Kernel/Filter**: Small matrix that scans the image
- **Stride**: How many pixels to move filter each step
- **Padding**: Border pixels added to control output size
- **Feature Map**: Output showing detected patterns

Transfer Learning

- **Concept**: Reuse pre-trained model on new, often smaller dataset
- **Process**: Keep early layers (generic features) + fine-tune last layers (task-specific)
- **Benefits**: Saves time, reduces data needed, often better performance on small datasets

Inception Network (Transfer Learning Example)

- **Architecture**: Uses parallel convolutions (Inception modules) for multi-scale feature capture
- **Pre-training**: Trained on ImageNet (1000 classes)
- **Use**: Common backbone in transfer learning workflows

Data Augmentation

- **Definition**: Create modified versions of training images (rotate, flip, crop, brightness, etc.)
- **Label**: Unchanged (rotated cat is still cat)
- **Effect**: Artificially enlarge dataset, reduce overfitting
- **When**: Applied to training data only (not validation/test)
- **Common Operations**: Flip, rotate, scale, crop, color jitter, noise

Object Detection

- **Task**: Find **where** (bounding box) and **what** (class) objects are in image
- **Output**: Multiple boxes + class labels + confidence scores per box
- **Differs from Classification**: Classification = single label per image; detection = multiple objects with locations

YOLO (You Only Look Once) - High Level

- **Type**: **One-stage, single-shot** detector
- **Process**: Single CNN forward pass on entire image
- **Grid Prediction**: Image divided into grid; each cell predicts boxes + object class + confidence
- **Speed**: Real-time capable (very fast)
- **Advantage**: Single pass (efficient), good accuracy
- **vs Two-Stage**: R-CNN family first proposes regions, then classifies (slower)

YOLO Key Points

- "One Look" = single forward pass
- Outputs: Bounding boxes + class probabilities + objectness scores
- Real-time detection
- Grid-based prediction system

Quick MCQ Decision Matrix

Activation Functions

- **Sigmoid/Tanh → Vanishing Gradient?** YES
- **ReLU → Vanishing Gradient?** NO (preferred for deep networks)
- **Output layer → Classification?** Sigmoid (0-1) for binary, Softmax for multi-class

Regularization Techniques

- **Prevents Overfitting?** Early Stopping, Dropout, L1, L2 - ALL YES
- **Creates Sparse Models?** L1 only
- **Used at Inference?** Only L1/L2 (dropout OFF)

Optimizers

- **Uses Momentum?** Momentum, Adam - YES
- **Adaptive Learning Rate?** Adam only
- **Most Popular Default?** Adam

Gradient Descent Variants

- **1 sample per update?** SGD
- **Full dataset per update?** Batch GD
- **Most practical/common?** Mini-batch GD

CNNs

- **Detects Local Patterns?** Convolutional layer
- **Good for Images?** CNNs
- **Reuses Pre-trained Model?** Transfer Learning
- **Single Forward Pass Detection?** YOLO

Common Exam Trick Questions

1. **Backpropagation ≠ Optimization**: Backprop computes gradients; gradient descent updates weights
2. **Dropout at Inference**: OFF (all neurons active), NOT on
3. **L1 vs L2**: L1 makes weights zero (sparse); L2 shrinks smoothly
4. **Mini-batch**: More practical than pure SGD or batch GD
5. **YOLO**: One-stage, single pass, real-time (NOT two-stage like R-CNN)
6. **Transfer Learning**: Reuse early layers, fine-tune last layers (NOT train from scratch)
7. **Dynamic vs Static Graph**: PyTorch is dynamic; TensorFlow (classic) is static
8. **Single-layer Network**: Cannot solve non-linearly separable problems (XOR)
9. **Autoencoder**: Unsupervised; goal is reconstruction, not classification
10. **Data Augmentation**: Applied to training data only; label stays same

---

Formula Cheat Sheet (Exam-Minimal)

Neuron

$z = w_1x_1 + w_2x_2 + ... + b$
$a = \text{activation}(z)$

SGD Weight Update (Plain)

$w_{new} = w_{old} - \eta \cdot (\partial L/\partial w)$

Regularization Terms (Conceptual)

- L1: Penalty $\propto \Sigma|w|$
- L2: Penalty $\propto \Sigma w^2$

Sigmoid

$\sigma(x) = 1 / (1 + e^{(-x)}) \rightarrow$ Range (0, 1)

ReLU

$f(x) = \max(0, x) \rightarrow$ Outputs 0 if $x \leq 0$, else x

Tanh

$\tanh(x) \rightarrow$ Range (-1, 1)

Last-Minute Review (5 min before exam)

✓ ReLU preferred in hidden layers (modern networks)
✓ Sigmoid/Tanh for output (binary classification)
✓ Early stopping + Dropout + L1/L2 all prevent overfitting
✓ Mini-batch is standard (not pure SGD, not batch)
✓ Adam = momentum + adaptive learning rate = most common
✓ YOLO = one-stage, one-pass, real-time object detection
✓ Transfer learning = reuse + fine-tune, not train from scratch

✓ Data augmentation = training only, label unchanged
✓ Backprop = gradient computation (paired with optimizer)
✓ PyTorch = dynamic, Keras = high-level API

Good Luck on CCEE Exam!
Remember: MCQs test concepts not memorization. Understand the "why", not just the "what".